

Root on Linux. DoS on Android.

What happens when a 99% reliable kernelCTF root exploit meets modern mobile defense-in-depth?

10-WEEK EMPIRICAL ASSESSMENT

102,740 Automated Race Attempts.
21 Dead Ends Documented.
0 Root Shells on ARM64 GKI.

The Target: Bad Epoll

CVE-2026-46242 is a Use-After-Free in **fs/eventpoll.c** discovered by Jaeyoung Chung via Google kernelCTF.

99%

X86_64 VM ROOT SHELL

0 / 102k

ANDROID GKI RACE WINS

The exploit easily pops root on a desktop VM. But when ported to a deliberately backported ARM64 Android GKI 6.1 testbed, every single escalation path collapsed.

How Root Was Won on x86

On unhardened Linux VMs, the exploit chain was fully deterministic.

STEP 01

Heap Spray via msg_msg

Reclaimed the dangling epitem allocation inside the `kmalloc-192` slab cache.

STEP 02

KASLR Defeat

Disclosed kernel text base pointers through uninitialized heap layout reads.

STEP 03

ROP to UID 0

Pivoted kernel stack into `commit_creds(&init_cred)` for instant root execution.

The First Barrier

125ns

The Natural Race Execution Window

Under Android's **CONFIG_PREEMPT_DYNAMIC=voluntary**, the `cond_resched()` call inside `eventpoll.c` is a complete no-op.

```
// __ep_remove execution window: // 250-550 CPU cycles (~125-275 ns @ 2.0  
GHz) // Natural thread scheduling: 0 hits in 102,740 attempts
```

Debugger watchpoints create an illusion of exploitability that disappears during real-time execution.

4 Chains. 21 Dead Ends.

Even under forced synthetic race wins, every single escalation path hit structural dead ends.

CHAIN 0

Refcount Decrement

`percpu_counter_dec` always locked onto the valid outer epoll structure, never the freed target.

CHAIN 1

Dual-Watch KASLR Leak

Structurally blocked by multi-watch sanity checks in `eventpoll.c:826`.

CHAIN 2 & 3

Fixed NULL Write

Reclaiming with `msg_msg` locked the decrement to `root_user`, leaving only an 8-byte NULL write at offset 160.

The Offset 160 Audit

Can zeroing 8 bytes at offset 160 in **kmalloc-192** yield anything other than an immediate kernel panic?

STRUCT	TARGET FIELD	OBSERVED RESULT
fib6_info	fib6_metrics	Panic: NULL route deref
snd_timer_user	queue link	Panic: Broken list traversal
packet_fanout	next member	Panic: Teardown pointer crash

Finding: Every reachable struct crashes immediately when offset 160 is zeroed. No privilege escalation primitive exists.

Why Mobile Defenses Won

Android's hardware and kernel mitigation stack dismantled all remaining fallback paths.

DEFENSE 01

PAC (Pointer Authentication)

Neutralized function pointer overrides and return address hijacking on physical ARM64 silicon.

DEFENSE 02

kCFI & BTI

Enforced forward-edge control flow integrity, blocking indirect branch pivots.

DEFENSE 03

Slab Isolation & Memory Latency

Cross-cache sprays were neutralized; physical memory bus latency broke heap reliability.

The Complete 26-Page Report

Exploit research isn't just about finding the 1% that pops shells—it's about rigorously proving why the 99% fail.

INCLUDED IN THE TECHNICAL DOSSIER

- Full Verification Ledger (VER-001 to VER-039)
- 21 Documented Dead Ends with killing evidence
- Complete Tier 1 offset tables & ROP chains
- Android GKI kernel harness & disassembly

Drop **"EPOLL"** in the comments below and I'll DM you the full 26-page PDF.